

Final Project Report

Charlie Argurst, Kathryn Bigelow, Arbri Cenolli
CIS 3200 - W24

Abstract

For our project, we tried multiple approaches to movie recommendations using the [MovieLens 20M](#) movie dataset on Kaggle. The dataset contains the features userID, movieID, titles, genres, and ratings. Our project highlights the comparison/effectiveness of our attempts at non-personalized recommendations, hybrid recommendations, and user-user collaborative filtering. After experimenting with each type of system, we found the simple algorithms to be the best, and settled on user-user collaborative filtering for our final system for testing and further analysis. We found that our collaborative filtering system had an RMSE of 0.93 between our predicted ratings and the users' actual ratings. Improvements to the system can certainly be made through the collection of additional user data or the modification of our system to include the movie genre information in the dataset.

Introduction

Recommendation systems are involved in almost all of the digital media we interact with daily. From product recommendations on Amazon to content recommendations on YouTube and even Netflix recommending movies. These systems utilize clever algorithms that can accurately and efficiently recommend items to users. Not only do these recommendation systems allow companies to efficiently reach more consumers, but they effectively improve user-platform interaction and experience. We were motivated to choose this subject to further explore and understand how these algorithms work and figure out how we could attempt various approaches to these types of systems ourselves.

Recommendations solve the problem of decision fatigue. It is a tool that navigates and narrows down the overwhelming array of options consumers face, providing personalized suggestions based on individual preferences, therefore simplifying the decision-making process and improving user satisfaction. There are infinitely many movies available to us, so it's not feasible to watch all the trailers and read a bunch of descriptions to figure out the movies we are most likely to rate favorably. Hence, we wanted a method to simplify this process and have the system do the dirty work for us.

There are many different approaches one could take to building a recommendation system, so we wanted to answer the question, "What recommendation system best identifies the movies that users will enjoy?" Specifically, we are proposing a recommendation system that can accurately recommend a movie to a user by exploring and analyzing various approaches of recommendation systems. We do this through the exploration of hybrid, non-personal, and collaborative filtering, finally settling on user based collaborative filtering as our simplest and most efficient system.

Related Work

Considering how important recommendation systems are in digital media, there is no shortage of work related to our focus question. *The Netflix Prize* was an open competition in

which people competed to create the best collaborative filtering algorithm for Netflix. The training data provided by Netflix consisted of users, movies, and ratings, similar to the data set we are using. The submitted algorithms were evaluated using root mean squared error (RMSE) and the one which gave a 10% improvement from Netflix's own algorithm was declared the winner. Another similar algorithm is discussed in the article, *How I made a Book Recommendation System using Python*, where collaborative filtering was utilized to give book recommendations given a book title.

In addition to these specific examples, we gained inspiration from the Datacamp course *Building Recommendation Engines in Python* as well as Chapter 9 of the book, *Mining of Massive Datasets*. These two resources provided a generic framework for the different recommendation systems we constructed, and our method for predicting user ratings mirrors the method demonstrated in the class slides

Methodology

We wanted to cover our bases and explore a variety of systems to find one that was both simple to implement and gave good recommendations. In order to work on our machines, we took a sample of 5,000 users and their movie ratings to perform our analysis. Our process initially started with attempting a complicated hybrid concept, as we thought this may produce the best results; however, we quickly discovered that simpler algorithms provided suitable results while being easy to understand and implement.

Hybrid Content/Collaborative Approach: We originally wanted to identify similar users based on the counts of the genres they watched, weighting the scores based on the popularity of the genre and then scaling the users' scores from 0 to 1. Once we found similar users, we wanted to use content-based recommendations to find the movies most similar to the user's profile from the pool of movies given by the similar users. However, not only did this algorithm take a while to produce recommendations, but we ultimately found it hard to implement and test.

Non-Personalized Approach: We then considered non-personalized methods to get a baseline for our system. Our non-personalized methods recommend movies based on aggregate statistics of the most popular movies in our dataset. To make recommendations, we identified movies that had the highest frequency of ratings and which had the highest average rating among users. For example, "The Shawshank Redemption" was rated highly and by many users, so it would be recommended to all users. However, we recognize that this defeats the purpose of a recommendation system in some sense, because these recommendations aren't tailored to the users' specific interests.

User-User Collaborative Filtering Approach: Thus, the final system we chose to build uses collaborative filtering to find similar users and predict scores of movies not watched by a given user. To start this process, we built a utility matrix with the user IDs as the rows, movie IDs as columns, and cells being the 1-5 rating given by the user. While the ratings were initially 1-5, we subtracted the users' mean from each value to center the data around 0. We then found the cosine similarity between all the users in this matrix. When recommending movies the system will find the 10 most similar users to a particular user and create a new utility matrix with those 10 users and their watched movies minus those seen by the target user. We then use a function to predict the scores for each movie in this new utility matrix using the similar users' ratings and their similarity scores to the target user.

For example, to predict user 4's rating for the movie, "Get Out". Out of the 10 similar users, this movie was only rated by 3 other users, user 126, 345, and 410. To calculate the predicted score, each of these 3 users' ratings is multiplied by their similarity score to user 4 and then added together, then divided by the sum of the similarity scores. This calculation is repeated for all other movies and the system returns the movies with the 5 highest predicted ratings. This process is then repeated for every user in our data and written to a .txt file. Overall, we decided that this was our best system due to its simplicity and accuracy considering the root mean squared error.

Experimental Discussion (Done)

We focused mainly on testing our final user-user collaborative filtering system using root mean squared error. To test the system we found the average RMSE between our system's predicted ratings and the actual ratings for a random sample of 10 users. For each user, we hid the ratings of the first 20% of movies in the data, which would become our test set. Given the sparsity of our data, we had to be careful to select our sample of users based on if our recommendation system was able to predict a rating for a movie within the range of movies we concealed. For a given user, we ran the recommendation system and collected the predicted ratings. We found the intersection of the sets of movies in our test set and that we had predicted scores for and found the RMSE for each user.

After collecting all of the RMSE scores for each user in our sample, we took a simple average. We discovered that increasing the number of similar users we used to predict scores from 5 to 10 slightly lowered the RMSE from 0.933 to 0.928. This means that our predictions have an error of about 0.93 stars.

Contribution

Our team met weekly to discuss the progress and actions steps for the project. In the initial stages of the project, each team member took over a different type of recommendation system so we could determine which system would be the best and most plausible to execute. Once we determined our final system method, we collaborated on writing the code and the report. Specific contributions are listed below:

Arbri focused on researching and coding non-personalized and content-based systems. He also wrote the abstract and introduction section of the report.

Charlie explored possible hybrid recommendation systems and assisted in writing the collaborative filtering system. He wrote the related work and methodology parts of the report.

Kathryn focused on the user-user collaborative filtering system along with final testing and consolidation of code. She wrote the experimental discussion, contribution, and conclusion parts of the report.

Conclusion

When we began our approach to recommendation systems, we were pretty ambitious in wanting to create a complicated system that would guarantee positive recommendations for our users. However, we quickly discovered that simpler algorithms were much better both in terms of their ease of implementation and quality of their recommendations. Collaborative filtering based on user similarities allowed us to leverage the quality judgment of our users, which we

found valuable given the limited amount of information and features in our data. While an error interval of about 1 star is reasonable for casual movie watchers, it may be too wide for users who want to be sure that they will enjoy a particular movie. Thus, the results of our testing show that we would need to make improvements to our system if we want to ensure that our recommendations are tailored to our users' tastes.

To make improvements to our system, we may look at tweaking our parameters to optimize the system we already have. This may be done through increasing the number of similar users we use to predict scores. However, the best thing we could do to improve our current system is to decrease the sparsity of our matrix by finding more data. We could either ask the users to watch more movies and make more ratings (which would be improbable in the real world) or add more features, like implicit data from user actions. Seeing as though our approach also suffers from the cold start user problem, our system may also be improved by using a hybrid of user collaborative filtering and content-based filtering of the movies given the genre information in the dataset. We may also improve the program's performance by using MapReduce methods to process the data and calculate the ratings for each user.

While our recommendation system may not be as sophisticated as those of Netflix or Amazon, it does offer a good foundation for users to find movies enjoyed by users similar to them. With a few additional components and more data we may be able to further optimize our recommendations to provide our users with the best movie-going experience possible.

References

- How I made a Book Recommendation System using Python.* (2023, May 23). Medium. Retrieved April 21, 2024, from <https://medium.com/@xaradxarma/book-recommendation-system-8cdb77585b65>
- Leskovec, J., Leskovec, J., Rajaraman, A., & Ullman, J. D. (2014). *Mining of Massive Datasets*. Cambridge University Press.
- Martin, B. (n.d.). *Python Pandas Tutorial: A Complete Introduction for Beginners*. LearnDataSci. Retrieved April 21, 2024, from <https://www.learndatasci.com/tutorials/python-pandas-tutorial-complete-introduction-for-beginners/>
- O'Callaghan, R., Peterson, A., & Matsui, M. (n.d.). *Building Recommendation Engines in Python*. DataCamp. Retrieved April 21, 2024, from <https://app.datacamp.com/learn/courses/building-recommendation-engines-in-python>
- "User-Based Collaborative Filtering In Python | Machine Learning." YouTube, uploaded by Grab N Go Info, 22 January 2022, https://www.youtube.com/watch?v=cxcFi3RDrEw&list=TLPQMTYwNDIwMjQp6WeQUHNqZA&index=2&ab_channel=GrabNGoInfo